SOMAIYA
VIDYAVIHAR

K J Somaiya Institute of Technology
(Formerly known as K J Somaiya Institute of Engineering and Information Technology)
An Autonomous Institute Permanently Affiliated to University of Mumbai

Somaiya
TRUST

Department of Information Technology

Cover page ASSIGNMENT Academic Year: 26-27 Class: SY IT Sem: III

Subject: Data Structures & Analysis. Course Code:

Name: Anushka Shettigar Roll No: BS2-36

ASSIGNMENT NO : 1/2 ✓		Date of Performance: 25/09/26	Date of Correction: 25/10/26
Rubrics	**Marks (20)**	**Grade**	**Signature of Faculty**
Participation and Involvement (5)			
Preparation and Knowledge (5)			
Accuracy and neatness of Documentation(5)			
Punctuality (5)			

Grades: Excellent – E (17 – 20)

Very Good – A (13-16)

Average – B (9-12)

Satisfactory – C (5-8)

Unsatisfactory – D (< - 5)

Assignment 2.

Q1 Develop and evaluate a Student Recommendation System in C/C++ using appropriate sorting and searching algorithms. The System should manage student records and rank students based on a recommendation score. Implement suitable sorting techniques to efficiently retrieve a specific student record. Compare the performance of the implemented algorithms for different dataset sizes based on execution time and number of comparisons and prepare a comparison table showing the performance of each algorithm. Based on the experimental results, determine the most suitabe sorting and searching algorithms for the system.

Ans

```
#include <iostream>
#include <string>
#include <vector>
#include <chrono>
#include <cstdlib>
#include <algorithm>

using namespace std;
using namespace std::chrono;

struct Student{
    int id;
    string name;
    float score;
};
long long comparisonsCount = 0;
```

DSA 2 : Searching Performance Comparison Table :

Searching Algorithm	Dataset Size	Execution Time	No. of comparisons.
Sequential Search	100	0.003 ms	50
Binary Search	100	0.0008 ms	6
Sequential Search	500	0.012 ms	250
Binary Search	500	0.0010 ms	8
Sequential Search	1000	0.024 ms	500
Binary Search	1000	0.0012 ms	9

```
comparisonsCount++;
int mid = low + (high - low) / 2;
if (arr[mid].score == targetScore) return mid;
if (arr[mid].score < targetScore) high = mid - 1;
else low = mid + 1;
}
return -1;
}

vector<student> generateDataset(int size) {
    vector<student> list(size);
    for (int i=0; i< size; i++) {
        list[i].id = 1000 + i;
        list[i].name = "student_" + to_string(i+1);
        list[i].score = (rand() % 1000) / 10.0;
    }
    return list;
}

int main(){
    srand(42);
    cout << "Student Recommendation System Benchmarking Complete\n";
    return 0;
}
```

```
void mergeSort(vector<student> &arr, int left, int
                right){
    if(left<right){
    int mid= left+(right-left)/2;
    mergeSort(arr, left, mid);
    mergeSort(arr, mid+1, right);
    merge(arr, left, mid, right);
}}
```

```
int partition(vector<Student> &arr, int low, int high){
    float pivot = arr[high].score;
    int i = low-1;
    for(int j=low; j<high; j++){
        comparisons(count++;
        if(arr[j].score>=pivot){
            i++;
            swap(arr[i], arr[j]);
        }}
    swap(arr[i+1], arr[high]);
    return i+1;
}
```

```
void quickSort(vector<Student> &arr, int low, int high)
{if(low<high){
    int pi = partition(arr, low, high);
    quickSort(arr, low, pi-1);
    quickSort(arr, pi+1, high);
}}
```

```
int binarySearch(const vector<Student>, &arr,
    float targetScore){
    int low = 0, high = arr.size()-1;
    while(low<=high){
```

Department of: - /Class /Sem. /Sub. /Academic Year:-

```
while (j>=0){
    comparisonCount++;
    if (arr[j].score < key.score){
        arr[j+1] = arr[j];
        j--;
    } else {
        break;
    } }
    arr[j+1] = key;
    } }

void merge (vector<Student> &arr, int left, int mid,
int right){
        int n1 = mid - left + 1;
        int n2 = right - mid;

        vector<Student> L(n1), R(n2);
        for(int i = 0;  i<n1; i++) L[i] = arr[left + i];
        for(int j=0;  j<n2; j++) R[j] = arr[mid+1+j];

int  i=0, j=0, k=left;
while (i<n1 && j<n2){
        comparisonsCount++;
        if (L[i].score >= R[j].score){
            arr[k++] = L[i++];
        } else {
            arr[k++] = R[j++];
        }}
        while (i<n1)  arr[k++]  = L[i++];
        while (j<n2)  arr[k++] = R[j++];
```

```
void resetCounter(){
    comparisonsCount=0;
}

Void BubbleSort (vector<Student> &arr){
    int n = arr.size();
  for (int i=0;  i<n-1; i++){
    for (int j=0; j<n-i-1; j++){
      comparisonCount++;
    if (arr[j].score < arr[j+1].score){
      swap (arr[j], arr[j+1]);
  }}}.

void selection Sort (vector <Student> &arr){
    int n = arr. size();
for (int i=0; i<n-1; i++){
    int MaxIdx = i;
    for (int j = i+1 ; j<n; j++){
      comparisonsCount ++;
    if (arr[j].score > arr[MaxIdx].score){
      maxIdx = j;
    }}
    swap(arr[i], arr[maxIdx]);
  }
}

void insertionSort (vector <Student> &arr){
    int n = arr.size();
    for (int i=1; i<n; i++){
    student key = arr[i];
    int j = i-1;
```